

DUOC UC

Escuela de Informática y Telecomunicaciones

Sistema de Gestión Integral para Ferretería

Informe Técnico de Arquitectura, Implementación y Pruebas

Asignatura: Full Stacks I

Profesor(a): Claudio Rojas Rodriguez

Integrantes:

Mario Meléndez

Franco Gonzalez

Índice

Introducción	3
Desarrollo	3
1. Contexto	4
2. Requerimientos	5
3. Casos de Uso	6
4. Arquitectura de Software	7
5. Base de Datos y Migraciones (Flyway)	9
6. Estándares de Desarrollo y Código Limpio	10
7. Pruebas Unitarias y Repositorio	11
8. Documentación de API (Swagger/Postman)	12
Conclusión	13

Introducción

La transformación digital ha impulsado a las organizaciones a modernizar sus procesos mediante el desarrollo de soluciones informáticas que permitan mejorar la eficiencia, optimizar la gestión de la información y facilitar la toma de decisiones. En este escenario, las arquitecturas basadas en microservicios se han consolidado como una alternativa ampliamente utilizada debido a su capacidad para desarrollar aplicaciones escalables, mantenibles y con una clara separación de responsabilidades.

El presente informe describe el desarrollo de un **Sistema de Gestión Integral para Ferretería**, implementado como solución a una problemática real relacionada con la administración de productos, inventario, ventas y usuarios. Para su construcción se emplearon tecnologías actuales del ecosistema Java y una arquitectura de microservicios que favorece la independencia de los distintos componentes del sistema.

En las siguientes secciones se presentan el contexto del problema de negocio, los requerimientos funcionales y no funcionales, los principales casos de uso, la arquitectura implementada, la descripción de los microservicios, las buenas prácticas de desarrollo aplicadas, la estrategia de persistencia de datos, las pruebas unitarias realizadas y la documentación de las API desarrolladas.

1. Contexto

La empresa administra diariamente procesos relacionados con la venta de productos, el control de inventario y la gestión de clientes. Actualmente, gran parte de estas actividades se realizan de forma manual o mediante herramientas que no se encuentran integradas entre sí, lo que dificulta mantener un control preciso sobre la información del negocio.

Esta situación provoca problemas como diferencias entre el inventario físico y el registrado, demoras en el proceso de venta, dificultades para conocer la disponibilidad real de los productos y una administración poco eficiente de la información. Además, la falta de un sistema centralizado limita el seguimiento de las operaciones y aumenta la posibilidad de cometer errores durante la gestión diaria.

Para dar solución a esta problemática, se desarrolló un **Sistema de Gestión Integral para Ferretería**, cuyo objetivo es digitalizar los principales procesos del negocio mediante una plataforma que permita administrar usuarios, productos, inventario, ventas y notificaciones de forma organizada y segura. Con esta solución se busca optimizar la gestión de la información, mejorar el control de las operaciones y proporcionar una herramienta que facilite el trabajo diario de la empresa.

2. Requerimientos

2.1 Requerimientos funcionales

- Registrar un nuevo usuario en el sistema (ms-auth: POST /auth/register).
- Autenticar un usuario y obtener un token JWT (ms-auth: POST /auth/login).

- Validar un token JWT para autorizar operaciones protegidas (ms-auth: POST /auth/validate, consumido por el BFF).
- Crear, listar, buscar (por nombre y por categoría), actualizar y desactivar productos (ms-product).
- Registrar el inventario inicial de un producto, indicando stock y stock mínimo por bodega (ms-inventory).
- Consultar el listado de productos con stock bajo (ms-inventory: GET /inventory/stock-bajo).
- Agregar o descontar stock de un producto (ms-inventory: PUT /inventory/producto/{id}/agregar y /descontar).
- Registrar una venta, validando la existencia del producto y descontando stock automáticamente (ms-sales).
- Consultar ventas por producto, por identificador, y obtener un resumen agregado de ventas (ms-sales).
- Generar y consultar notificaciones de stock bajo y de ventas registradas (ms-notification).
- Marcar notificaciones como leídas, individualmente o todas en conjunto (ms-notification).
- Exponer todas las operaciones anteriores a través de un único punto de entrada (BFF), validando el JWT antes de reenviar la solicitud al microservicio correspondiente.

2.2 Requerimientos no funcionales

- Seguridad: autenticación basada en JWT (librería JWT 0.11.5) con contraseñas cifradas mediante BCryptPasswordEncoder.
- Disponibilidad y despliegue: contenerización completa con Docker y Docker Compose, incluyendo healthcheck de PostgreSQL.
- Documentación de API: especificación OpenAPI expuesta y documentación interactiva Swagger UI en cada servicio.
- Persistencia controlada: versionado de esquema de base de datos mediante Flyway en cada microservicio.
- Base de datos relacional independiente por servicio: PostgreSQL 16, con el patrón Database per Service.
- Mantenibilidad: separación en capas Controller / Service / Repository / DTO en cada microservicio, con manejo centralizado de excepciones.
- Escalabilidad: cada microservicio puede escalarse y desplegarse de forma independiente al ejecutarse como contenedor propio.

3. Casos de Uso

Caso de Uso 1: Registrar usuario

1. El usuario ingresa un nombre de usuario y una contraseña.
2. El sistema valida que el usuario no exista previamente.
3. La contraseña es cifrada mediante BCrypt.

4. El usuario queda registrado correctamente en la base de datos.
-

Caso de Uso 2: Iniciar sesión

1. El usuario ingresa sus credenciales.
 2. El sistema valida el nombre de usuario y la contraseña.
 3. Si las credenciales son correctas, se genera un token JWT.
 4. El usuario utiliza dicho token para acceder a los servicios protegidos.
-

Caso de Uso 3: Gestionar productos

1. El usuario autenticado accede al módulo de productos.
 2. El sistema valida el token JWT.
 3. El usuario puede crear, consultar, buscar, actualizar o desactivar productos.
 4. Los cambios son almacenados en la base de datos del microservicio de productos.
-

Caso de Uso 4: Gestionar inventario

1. El usuario autenticado selecciona un producto.
 2. El sistema valida el token JWT.
 3. El usuario puede registrar el stock inicial o modificar el stock existente.
 4. El sistema actualiza la información del inventario y permite consultar productos con stock bajo.
-

Caso de Uso 5: Registrar venta

1. El usuario autenticado registra una venta.
 2. El sistema valida el token JWT.
 3. Se verifica que el producto exista y tenga stock disponible.
 4. El sistema registra la venta y descuenta automáticamente el stock.
 5. Si el stock queda bajo el mínimo configurado, se genera una notificación.
-

Caso de Uso 6: Consultar notificaciones

1. El usuario autenticado accede al módulo de notificaciones.
2. El sistema valida el token JWT.
3. Se muestran las notificaciones generadas por ventas o stock bajo.

4. El usuario puede marcar una o todas las notificaciones como leídas.

4. Arquitectura de Software

El sistema fue desarrollado utilizando una **arquitectura de microservicios**, donde cada servicio cumple una responsabilidad específica y puede desplegarse de forma independiente. Para facilitar la comunicación entre el cliente y los distintos microservicios, se implementó un **Backend For Frontend (BFF)**, el cual actúa como único punto de entrada al sistema mediante el puerto **8080**.

El BFF recibe todas las solicitudes realizadas por el cliente, valida el token JWT mediante el microservicio de autenticación (**ms-auth**) y, posteriormente, redirige la petición al microservicio correspondiente. De esta forma, el cliente no necesita conocer la ubicación ni el funcionamiento interno de cada servicio.

La lógica de negocio se encuentra distribuida en los siguientes microservicios:

- **ms-auth**: administra el registro de usuarios, autenticación y validación de tokens JWT.
- **ms-product**: gestiona la información de los productos.
- **ms-inventory**: administra el inventario y el control de stock.
- **ms-sales**: registra las ventas y coordina las operaciones relacionadas con el proceso de venta.
- **ms-notification**: administra las notificaciones generadas por el sistema.

Cada microservicio fue desarrollado con **Spring Boot 3.3.4**, posee su propia base de datos PostgreSQL siguiendo el patrón **Database per Service**, administra sus migraciones mediante **Flyway** y puede ejecutarse de manera independiente utilizando **Docker Compose**.

La comunicación entre los microservicios se realiza mediante servicios **REST**, utilizando **RestClient** de Spring Framework para el intercambio de información entre ellos.

A continuación, se presentan los diagramas C4 correspondientes a la arquitectura del sistema.

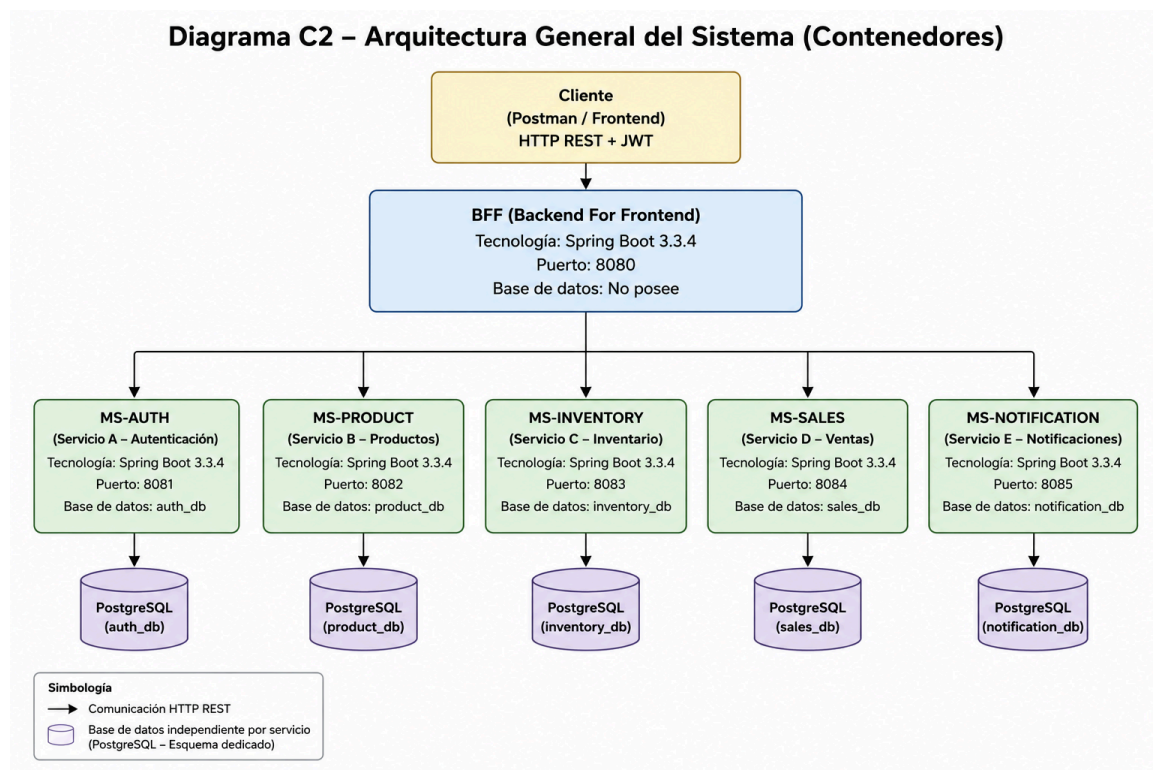
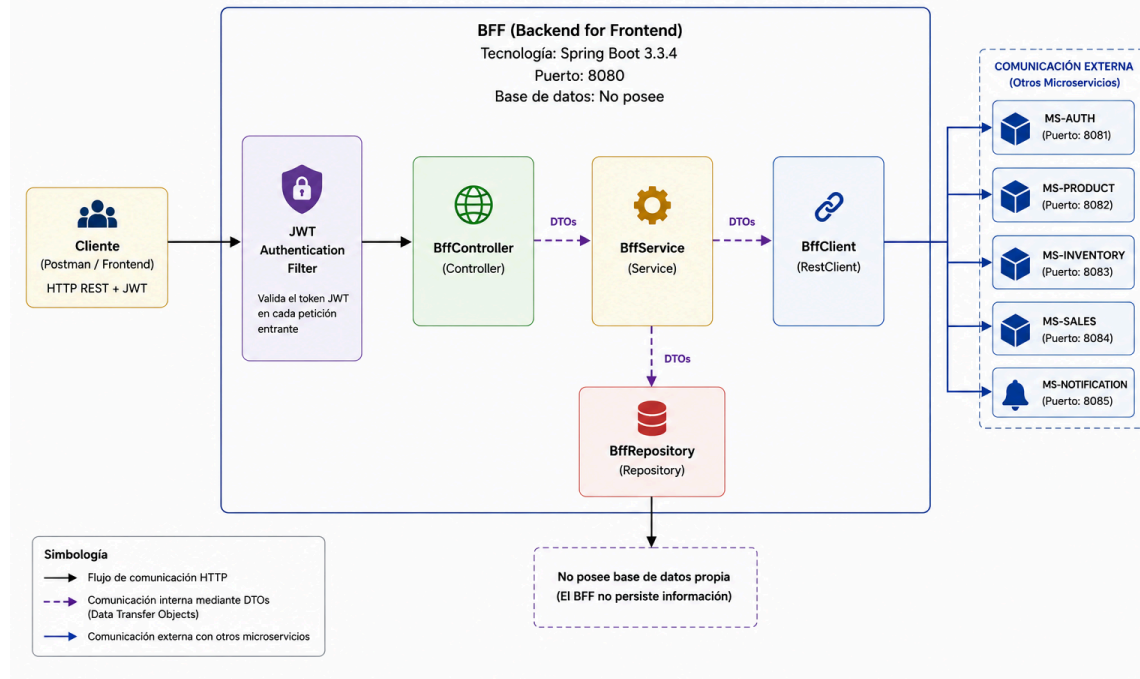


Diagrama C3 – Componentes Internos del BFF (Backend for Frontend)

Microservicio: BFF (Backend for Frontend)



5. Base de Datos y Migraciones (Flyway)

El proyecto implementa el patrón **Database per Service**, donde cada microservicio que requiere persistencia administra su propia base de datos PostgreSQL de forma independiente. Esta estrategia permite mantener una adecuada separación de responsabilidades, evitando el acoplamiento entre los distintos servicios y facilitando su mantenimiento y escalabilidad.

Para el control de versiones del esquema de base de datos se utilizó **Flyway**, permitiendo gestionar las migraciones de manera ordenada y reproducible. Cada microservicio incorpora una migración inicial (V1_...) encargada de crear la estructura necesaria para su funcionamiento.

Siguiendo las buenas prácticas de desarrollo, las migraciones contienen únicamente instrucciones de definición de estructuras (DDL), evitando mezclar la creación de tablas con la inserción de datos de prueba (DML). De esta manera, el despliegue del sistema puede realizarse de forma consistente en cualquier entorno.

Microservicio	Migraciones	Contenido
ms-auth	V1	Tabla usuarios (id UUID, username único, password, created_at, updated_at).
ms-product	V1 (DDL), V2 (DML)	V1 crea la tabla products. V2 inserta 8 productos de ejemplo (taladro, serrucho, martillo, llave inglesa, destornillador, cemento, pintura, cinta métrica).
ms-inventory	V1 (DDL), V2 (vacía)	V1 crea la tabla inventory con restricción CHECK de stock no negativo. V2 no contiene datos: el inventario inicial se carga vía API.
ms-sales	V1	Tabla ventas (product_id, cantidad, precio_unit, total, cliente, restricción CHECK de cantidad positiva).
ms-notification	V1	Tabla notifications (tipo, mensaje, product_id, leida, created_at).

6. Pruebas Unitarias y Repositorio (GitHub)

- . Para asegurar el correcto funcionamiento de la aplicación se desarrolló una estrategia de pruebas unitarias en todos los microservicios del proyecto. Las pruebas fueron implementadas utilizando **JUnit**

5, Mockito y H2 como base de datos en memoria para los escenarios de prueba, mientras que **JaCoCo** se empleó para generar reportes de cobertura del código.

- La estructura de pruebas considera distintos niveles de validación, incluyendo pruebas de carga del contexto de Spring (ApplicationTests), pruebas de controladores (ControllerTests), pruebas de servicios (ServiceTests), pruebas de repositorios (RepositoryTests) y validaciones de los DTO mediante Bean Validation. Esta estrategia permitió verificar el comportamiento esperado de cada componente de forma independiente.
- El proyecto fue gestionado mediante **Git** y alojado en **GitHub**. Como estrategia de trabajo se utilizó una rama principal (**main**) para mantener la versión estable del proyecto y ramas de desarrollo (**feature/**) para implementar nuevas funcionalidades antes de integrarlas a la rama principal. Esta metodología permitió mantener un historial organizado de cambios y facilitar el trabajo colaborativo durante el desarrollo.

Microservicio	ApplicationTests	ControllerTest	ServiceTest	RepositoryTest	DTO ValidationTest	JaCoCo
bff	Sí	Sí (6 controllers)	Sí (5 services)	N/A (sin BD)	Sí	Sí
ms-auth	Sí	Sí	Sí	Sí	Sí	Sí
ms-product	Sí	Sí	Sí	Sí	Sí	Sí
ms-inventory	Sí	Sí	Sí	Sí	Sí	Sí
ms-sales	Sí	Sí	Sí	Sí	Sí	Sí
ms-notificación	Sí	Sí	Sí	Sí	Sí	No

9. Documentación de API (Swagger/Postman)

Todos los microservicios incorporan documentación automática de sus endpoints mediante **OpenAPI/Swagger**, permitiendo visualizar la información de cada servicio, conocer los parámetros requeridos y realizar pruebas directamente desde la interfaz web.

Como parte del proceso de entrega, la especificación OpenAPI de cada microservicio fue exportada e importada en **Postman**, donde se organizó una colección de pruebas que facilita la validación del funcionamiento del sistema durante la demostración práctica. Esta colección permite ejecutar de forma ordenada operaciones como el registro de usuarios, autenticación, gestión de productos, administración de inventario, registro de ventas y consulta de notificaciones.

Servicio	Swagger UI	OpenAPI JSON
BFF	http://localhost:8080/swagger-ui.html	http://localhost:8080/api-docs
ms-auth	http://localhost:8081/swagger-ui.html	http://localhost:8081/api-docs
ms-product	http://localhost:8082/swagger-ui.html	http://localhost:8082/api-docs
ms-inventory	http://localhost:8083/swagger-ui.html	http://localhost:8083/api-docs
ms-sales	http://localhost:8084/swagger-ui.html	http://localhost:8084/api-docs
ms-notification	http://localhost:8085/swagger-ui.html	http://localhost:8085/api-docs

Conclusión

El desarrollo del Sistema de Gestión Integral para Ferretería permitió aplicar los principales conceptos relacionados con el desarrollo de aplicaciones distribuidas mediante una arquitectura de microservicios. La solución implementada aborda la problemática inicial del negocio, digitalizando procesos fundamentales como la gestión de usuarios, productos, inventario, ventas y notificaciones, mejorando el control de la información y facilitando la administración de las operaciones.

Durante el desarrollo del proyecto se incorporaron buenas prácticas de ingeniería de software, incluyendo una arquitectura desacoplada, persistencia independiente por microservicio, control de versiones de base de datos mediante Flyway, autenticación basada en JWT, documentación de API con OpenAPI/Swagger y una estrategia de pruebas unitarias para validar el comportamiento de los distintos componentes del sistema.

Como resultado, se obtuvo una solución escalable, mantenible y preparada para futuras ampliaciones, cumpliendo con los objetivos planteados y con los lineamientos técnicos establecidos para el desarrollo del proyecto.